# Thông tin chung

## PRACTICAL EXAMINATION

Exam Duration: 85 minutes

### Instructions

**Please read these instructions carefully before answering the questions:**

- You may use the materials on your computer, notebooks, and textbooks.
- You are NOT permitted to use electronic devices or other methods to share data with others.
- Do not randomly disconnect the FPTU_EXAMONLINE network during the test. If any network problems occur, immediately notify the exam supervisor for assistance.

**In addition to the conditions above, students must also fulfill the following requirements:**

- Use Visual Studio Code (VS Code), MongoDB Compass, Postman or Thunder Client (a VSCode extension) tools (offline) to complete the exam.
- Use the NodeJS platform version 22.11.0.
- Use the browser software available on your computer.

**This part is very important — read it carefully and follow the instructions:**

- You are given the data files in the provided directory.
- You must use the solution sent in advance via email and install the additional library included in the accompanying request.
- You are not allowed to download any additional libraries during the test.
- Violating any of the above will render your exam invalid.

### System Requirements

Use MongoDB Compass to create a database named "**su26db**", create collections matching the data in the given directory, and then import the data into those collections.

Based on the given materials, build a RESTful API application that fulfills the requirements of the problem.

**Note: 0 will be given to the work that:**

- Not using connection string to MongoDB in the .env file.
- Not configuring the root path of RESTful API Web App at: http://localhost:9999

**All endpoints must:**

- Validate input data.
- Apply business rules.
- Return appropriate HTTP status codes.
- Return properly structured JSON responses.

### Context

This system is a Student Attendance Management platform. Its data is organized into four collections: **students**, **classes**, **subjects**, and **attendances**.

Students must build the required API endpoints using ExpressJS, MongoDB, and Mongoose. Each task specifies an API operation to implement, covering input validation, business rules, database processing, and proper JSON response formatting. Students must test their APIs with Postman (offline) or Thunder Client.

---

# Question 1 (1.5 points): Retrieve Students by Class

**Objective:** Return the list of students enrolled in a given class.

**Endpoint:**

```
GET /api/classes/:classId/students?status=enrolled
```

**Business Rules:**

- Validate classId as a valid ObjectId. Return 400 if invalid.
- Return 404 if class is not found.
- Only accept valid status values: enrolled or dropped. Return 400 if missing or invalid.
- Output fields: id (renamed from _id), studentCode, fullName, email.

**Expected Response:**

**- 400 (Bad Request): Invalid ObjectId**

```
{
```

```
}
```

- 404 (Not Found)

— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —

```
{
    "error": "Class with id = 65bb000000000000000099 not found"
}
```

- 400 (Bad Request): Invalid status

```
{
    "error": "Invalid status: active"
}
```

- 200 (OK): Test with classId = "65bb00000000000000000001"

```
[
    {
        "id": "65dd00000000000000000001",
        "studentCode": "SE001",
        "fullName": "Nguyen Van A",
        "email": "anvn@fpt.edu.vn"
    },
    ......
]
```

---

## Question 2 (2.5 points): Retrieve Attendance Statistics by Class

**Objective:** Return the attendance summary for all students in a given class.

**Endpoint:**

```
GET /api/classes/:classId/attendance-summary
```

**Business Rules:**

- Validate classId as a valid ObjectId. Return 400 if invalid.
- Return 404 if class is not found.
- For each student enrolled in the class, calculate:
  - totalSessions: total number of attendance sessions recorded for that student.
  - present: number of PRESENT records.
  - absent: number of ABSENT records.
  - attendanceRate (%): (present / totalSessions) * 100, rounded to 1 decimal.
  - warning: true if attendanceRate < 80%, otherwise false.

**Expected Response:**

- 400 (Bad Request)

```
{
    "error": "Invalid classId: 65bb000000000000000000"
}
```

- 404 (Not Found)

```
{
    "error": "Class with id = 65bb000000000000000099 not found"
}
```

- 200 (OK): Test with classId = "65bb00000000000000000001"

```
[
    {
        "studentCode": "SE001",
        "fullName": "Nguyen Van A",
        "totalSessions": 10,
        "present": 9,
        "absent": 1,
        "attendanceRate": 90.0,
```

```
        "warning": false
    },
    {
        "studentCode": "SE002",
        "fullName": "Tran Thi B",
        "totalSessions": 10,
        "present": 7,
        "absent": 3,
        "attendanceRate": 70.0,
        "warning": true
    },
    ......
]
```

---

## Question 3 (2.5 points): Retrieve Attendance History of a Student

**Objective:** Return the full attendance history of a specific student across all classes.

**Endpoint:**

```
GET /api/students/:studentId/attendances
```

**Business Rules:**

- Validate studentId as a valid ObjectId. Return 400 if invalid.
- Return 404 if student is not found.
- For each attendance record, populate:
  - className: from the classes collection.
  - subjectName: from the subjects collection (via class).
- Sort results by date descending (most recent first).

**Expected Response:**

**- 400 (Bad Request)**

```
{
    "error": "Invalid studentId: 65dd000000000000000000"
}
```

**- 404 (Not Found)**

```
{
    "error": "Student with id = 65dd000000000000000099 not found"
}
```

**- 200 (OK): Test with studentId = "65dd00000000000000000001"**

```
[
    {
        "id": "65cc0000000000000000005",
        "className": "SE1701",
        "subjectName": "Project Management",
        "date": "2026-03-15T00:00:00.000Z",
        "status": "PRESENT"
    },
    {
        "id": "65cc0000000000000000003",
        "className": "SE1701",
        "subjectName": "Project Management",
        "date": "2026-03-08T00:00:00.000Z",
        "status": "ABSENT"
    },
    ......
]
```

---

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

## Question 4 (3.5 points): Bulk Attendance Entry with Upsert

**Objective:** Create or overwrite attendance records for all students in a class on a specific session date.

**Endpoint:**

```
POST /api/classes/:classId/attendances
```

Request Body:

```
{
    "date": "2026-03-15",
    "records": [
        { "studentId": "65dd0000000000000000000001", "status": "PRESENT" },
        { "studentId": "65dd0000000000000000000002", "status": "PRESENT" },
        { "studentId": "65dd0000000000000000000003", "status": "ABSENT" },
        ......
    ]
}
```

Business Rules:

- Validate classId as a valid ObjectId. Return 400 if invalid.
- Return 404 if class not found.
- Return 400 if date is missing or is a future date.
- Return 400 if records is missing or empty.
- The records array must include ALL students enrolled in the class - no partial submissions. Return 400 if any enrolled student is missing from the records.
- Each studentId must belong to the class. Return 400 if any studentId is not a member.
- Only accept status values: PRESENT or ABSENT. Return 400 if invalid.
- If attendance records already exist for the given date, overwrite them (upsert behavior - do not return an error).
- Return totalRecords, present, absent, and updatedCount (number of records that were overwritten).

Expected Response:

- 400 (Bad Request): Future date

```
{
    "error": "Date cannot be in the future"
}
```

- 400 (Bad Request): Missing student in records

```
{
    "error": "Missing attendance for student SE003 (Nguyen Van C)"
}
```

- 400 (Bad Request): Student not in class

```
{
    "error": "Student 65dd000000000000000099 does not belong to class"
}
```

- 201 (Created): Test with classId = "65bb000000000000000000001"

Example: a fresh submission (no records exist yet for that date)

```
{
    "message": "Attendance recorded successfully",
    "totalRecords": 3,
    "present": 2,
    "absent": 1,
    "updatedCount": 0
}
```

Example: a re-submission (existing records are overwritten)

```
{
    "message": "Attendance recorded successfully",
    "totalRecords": 3,
    "present": 2,
    "absent": 1,
    "updatedCount": 3
}
```

---- END ----